Assignment 2

Warehouse Robot Decision Making with Dynamic Programming and Reinforcement Learning

CPSC-4117EL-01 Artificial Intelligence | Fall 2026
Group 12: Leyan Zeng (0480234), Lanhui Zhang (0465267)

1. Introduction

This assignment studies a warehouse robot operating in a 10 × 10 grid. The robot must collect a package from the pickup station, deliver it to the packing station, and return to the charging dock. Shelves block movement, while congestion zones remain traversable but carry a penalty.

The task is formulated as a Markov Decision Process (MDP). Value Iteration is used when the transition and reward models are known, while Q-Learning learns action values through interaction with the environment. The report follows the assignment structure: MDP formulation, the two algorithms, experiments under different motion probabilities, and a comparison of their performance.

2. MDP Formulation

2.1 State Space and Markov Property

The state is defined as s = (row, col, has_package, delivered, steps_remaining) = (r, c, h, d, τ). The row and column identify the robot location; h indicates whether the package has been collected; d indicates whether it has been delivered to the packing station; and τ records the number of steps remaining before the 120-step limit. The remaining-step variable is part of the state for both Value Iteration and Q-Learning.

State component	Meaning
row, col	Robot position on a non-shelf cell.
has_package ∈ {0, 1}	0 before pickup; 1 after pickup.
delivered ∈ {0, 1}	0 before delivery; 1 after delivery.
steps_remaining ∈ {0, …, 120}	Remaining steps before the maximum-step terminal condition.

The task-progress combinations used are (0, 0), (1, 0), and (1, 1). The combination (0, 1) is not valid because delivery cannot occur before pickup.

Robot position alone is not sufficient. At the packing station, the delivery reward is available only if the robot is carrying the package, and the return reward is available only after delivery. The flags h and d preserve this task progress. The remaining-step variable τ is also necessary because the set of achievable future rewards depends on how many steps are left. For example, the same position and task-progress flags can require different actions when τ is large versus when τ is close to zero. Including these variables makes the next-state distribution, reward, and termination behaviour depend on the current state and action rather than on the full episode history.

Formally, let F denote the set of non-shelf cells. The finite-horizon state space is S = {(r, c, h, d, τ) : (r, c) ∈ F, (h, d) ∈ {(0, 0), (1, 0), (1, 1)}, τ ∈ {0, 1, …, 120}}. The terminal-state set is S_T = {s : τ = 0} ∪ {s : (r, c) = dock and d = 1}. There are 27,951 valid state combinations for the fixed layout: 77 passable cells × 3 task-progress phases × 121 remaining-step values.

2.2 Action Space and Transition Model

The action space is A = {up, down, left, right}. In the baseline setting, the intended direction occurs with probability 0.8 and each perpendicular direction occurs with probability 0.1. For example, when the robot chooses up, it moves up with probability 0.8, left with probability 0.1, and right with probability 0.1. If a movement would enter a shelf or leave the grid, the robot stays in its current cell and receives the invalid-move penalty in addition to the step penalty. Congestion cells are traversable but incur a penalty.

For a selected action a, let $a^{\perp}1$ and $a^{\perp}2$ denote the two perpendicular directions. The three possible directional outcomes have probabilities p, $(1 - p)/2$, and $(1 - p)/2$, respectively. Each outcome is passed through the environment movement rule: a move into a shelf or outside the grid leaves the robot in its current cell. If multiple directional outcomes lead to the same next state, their probabilities are added to obtain $P(s' \mid s, a)$. On every transition, the remaining-step component decreases by one: $\tau' = \tau - 1$.

The task-progress flags are updated by the resulting position. When the robot reaches the pickup station, h changes from 0 to 1. When it reaches the packing station while h = 1 and d = 0, d changes from 0 to 1. Otherwise, the flags remain unchanged. Therefore, the valid task-progress sequence is (0, 0) → (1, 0) → (1, 1), and delivery cannot occur before pickup.

2.3 Reward Model

Event	Reward
Pick up the package	+25
Deliver the package at the packing station while carrying it	+40
Return to the charging dock after delivery	+100
Enter a congestion zone	−10
Each time step	−1
Attempt to move into a shelf or outside the grid	−3 additional penalty

The reward is transition-based and additive. The ordinary step penalty applies on every transition, and event-specific rewards or penalties are added when the corresponding event occurs. The invalid-move penalty is additional to the step penalty. The congestion check follows the supplied environment: it is applied to the resulting cell after blocked movement has been resolved.

Using an indicator I_{event} that equals 1 when an event occurs on the transition and 0 otherwise, the reward is $R(s, a, s') = -1 + 25I_{pickup} + 40I_{delivery} + 100I_{return} - 10I_{congestion} - 3I_{invalid}$. $I_{invalid}$ is 1 when the attempted move is blocked by a shelf or grid boundary. $I_{congestion}$ is 1 when the resulting cell after movement resolution is a congestion cell. The −3 invalid-move penalty is therefore added to the ordinary −1 step penalty.

2.4 Discount Factor and Terminal Conditions

The discount factor is $\gamma = 0.95$. An episode terminates after the robot has delivered the package and returns to the charging dock, or when τ reaches zero. The transition that completes the return or uses the final available step still receives its immediate reward; terminal states have zero future value. The supplied grid layout is fixed with random seed 42 for the reported experiments; the pickup station is at (2, 2), the packing station at (0, 4), and the charging dock at (6, 0).

3. Part 1: Value Iteration

3.1 Algorithm and Implementation

Value Iteration estimates the best expected discounted return for every five-dimensional state. For terminal states, $V(s) = 0$. For each nonterminal state $s = (r, c, h, d, \tau)$, the Bellman optimality equation is:

$$\mathbf{V(s) = \max_a \Sigma_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V(s')], \quad s \notin S_T}$$

The implementation initializes the value of every valid state to zero and performs in-place (Gauss–Seidel) Bellman updates. States are processed in ascending order of τ, starting with terminal states at $\tau = 0$. Because every transition decreases τ by exactly one, the successor state of a state with τ steps remaining has already been updated in the current sweep. This ordering performs the finite-horizon recursion by backward induction. After each sweep, the maximum update size is calculated as $\delta = \max_s |V_new(s) - V_old(s)|$. The iteration stops when $\delta < \theta = 10^{-6}$. The greedy policy is extracted using the same expected-return expression, selecting the action with the highest value for each nonterminal five-dimensional state.

$$\text{Policy extraction: } \pi^*(s) = \operatorname{argmax}_a \Sigma_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V(s')], \quad s \notin S_T.$$

3.2 Implementation Settings and Convergence

Parameter	Setting
Layout seed	42
Maximum episode length	120 steps
Discount factor (γ)	0.95
Convergence threshold (θ)	0.000001
State representation	(row, col, has_package, delivered, steps_remaining)
Valid state combinations	27,951 (77 passable cells × 3 task phases × 121 time values)

Value Iteration is model-based. For every nonterminal state and action, it calculates the expected return over possible next states and therefore requires both $P(s' \mid s, a)$ and $R(s, a, s')$. Unlike Q-Learning, it does not estimate these probabilities from trial-and-error interaction. The supplied environment executes one deterministic direction per step; the Value Iteration implementation constructs the known stochastic model by enumerating the intended and two perpendicular movement outcomes, applying the environment movement and reward rules, and aggregating probabilities when outcomes produce the same next state.

3.3 Policy Visualizations and Results across Motion Probabilities

The updated five-dimensional Value Iteration policy is defined for every valid remaining-step value. Figures 1–3 show three selected policy slices: before pickup at $\tau = 120$, while carrying the package at $\tau = 105$, and after delivery at $\tau = 90$. Within each panel, τ is fixed so that the policy can be visualized on the two-dimensional grid.

Intended-action probability	Convergence sweeps	γ	θ
$p = 0.80$	2	0.95	1×10^{-6}

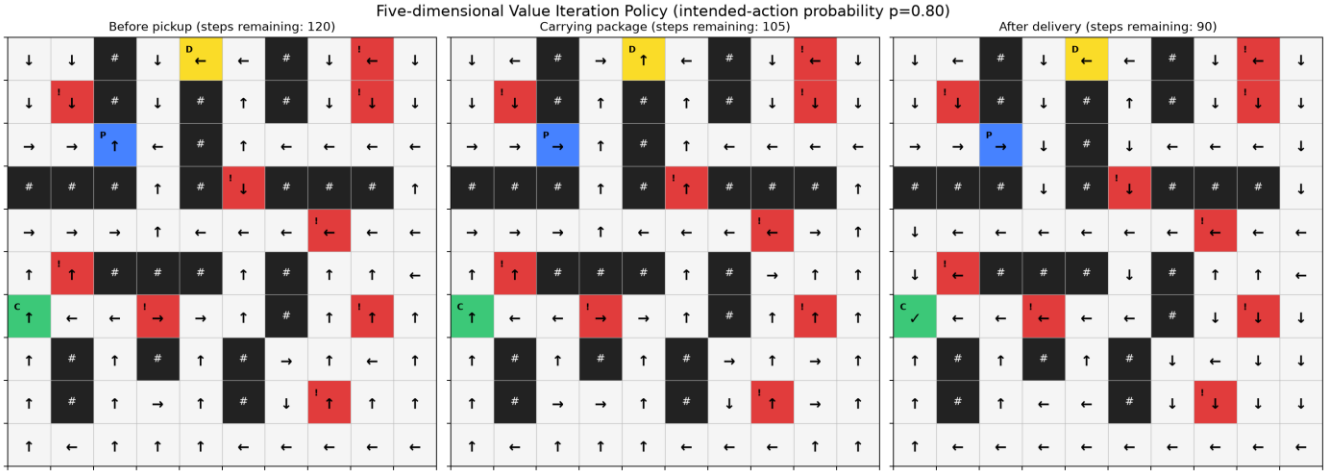


Figure 1. Five-dimensional Value Iteration policy at p = 0.80, showing the three task-progress phases at selected remaining-step values.

Figures 2 and 3 show the corresponding five-dimensional policies at p = 0.90 and p = 1.00, respectively, using the same layout and remaining-step slices.

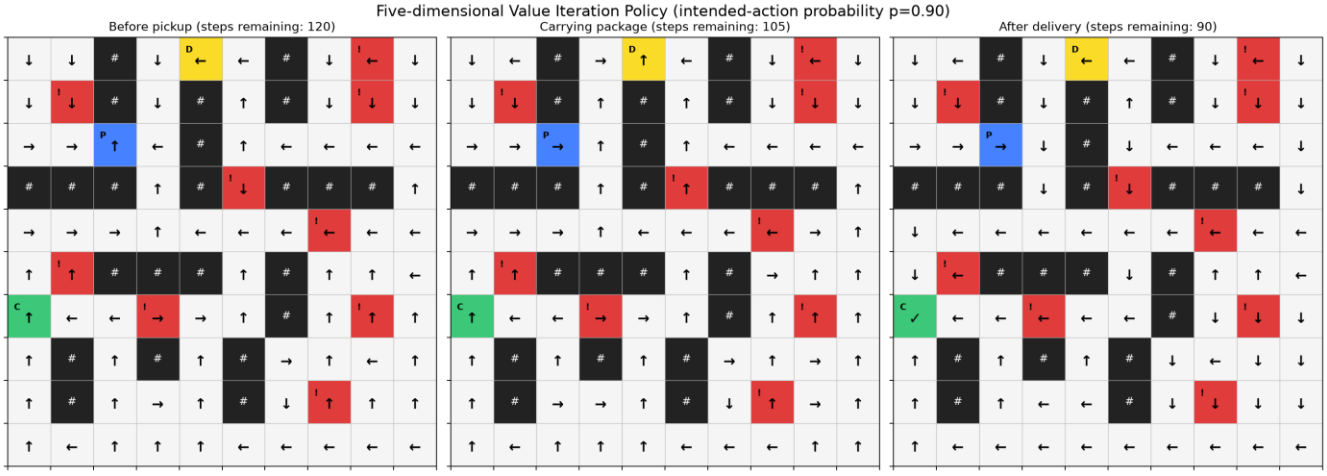


Figure 2. Five-dimensional Value Iteration policy at p = 0.90, showing the three task-progress phases at selected remaining-step values.

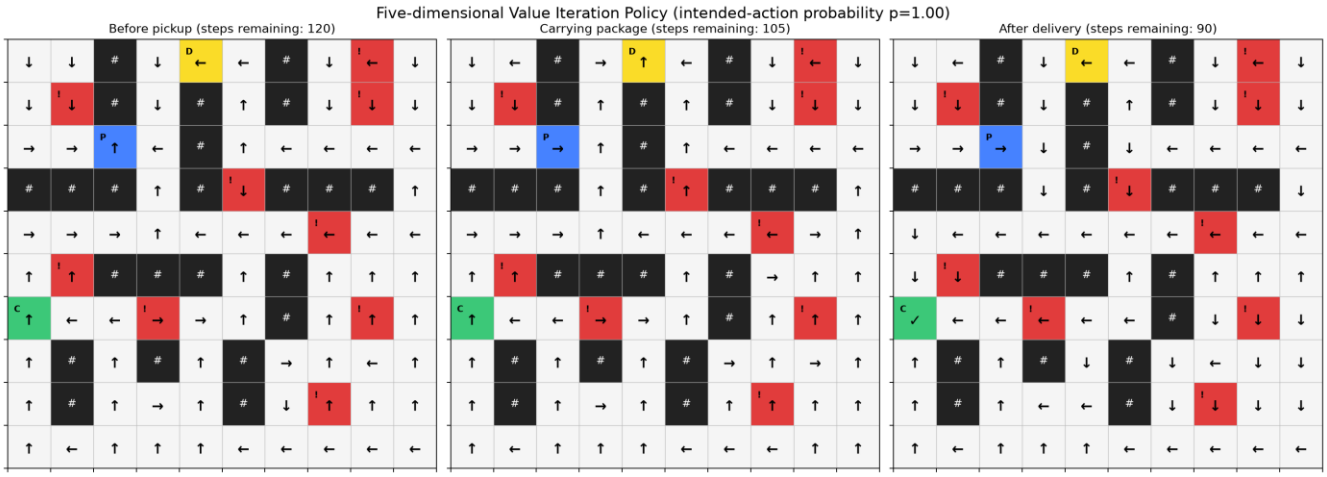


Figure 3. Five-dimensional Value Iteration policy at p = 1.00, showing the three task-progress phases at selected remaining-step values.

4. Part 2: Q-Learning

4.1 Algorithm and Implementation

Q-Learning estimates an action value for each state and action through repeated interaction with the supplied warehouse environment. The state is (row, col, has_package, delivered, steps_remaining), consistent with the MDP formulation in Section 2. The implementation explicitly initializes the Q-table to zero for 27,951 traversable, time-aware states over the three valid task-progress phases. Each state has four action values for up, down, left, and right, including actions that may attempt an invalid movement.

The agent updates its action values using the terminal-aware Q-Learning target. For a transition (s, a, r, s'), the target is $y = r$ when the episode ends; otherwise, $y = r + \gamma \max_{a'} Q(s', a')$. The update is $Q(s, a) \leftarrow Q(s, a) + \alpha[y - Q(s, a)]$. Thus, the future-value term is zero after either successful return or the 120-step limit. Epsilon-greedy exploration selects a random action with probability ε and otherwise selects an action with the highest current Q-value. Epsilon begins at 1.00, decreases after every training episode, and does not fall below 0.05. The agent observes sampled states and rewards; the intended-action probability is used by the environment adapter and is not passed to the Q-Learning update.

4.2 Hyperparameter Configurations and Random Seeds

Two hyperparameter configurations were tested. Configuration A uses a higher learning rate and faster exploration decay. Configuration B uses a lower learning rate, a higher discount factor, and slower exploration decay.

Parameter	**A: fast decay**	**B: slow decay**
Learning rate (α)	0.20	0.10
Discount factor (γ)	0.95	0.99
Initial epsilon	1.00	1.00
Epsilon decay	0.995	0.999
Minimum epsilon	0.05	0.05

For each intended-action probability $p = 1.00$, 0.90, and 0.80, both configurations were trained with seeds 11, 42, and 99. Each run used 5,000 training episodes and a maximum of 120 steps per episode. This produced 18 trained Q-tables and 90,000 training episodes. Each trained policy was evaluated for 100 additional episodes with exploration disabled, producing 1,800 evaluation episodes. The station layout remained fixed at seed 42; the episode seeds changed motion outcomes without regenerating the stations. The two configurations change several parameters together, so their differences cannot be attributed to one parameter alone.

4.3 Training Results and Convergence

The program records the cumulative reward, success indicator, episode length, and epsilon value for every training episode. Figures 4 and 5 show the 100-episode trailing averages, pooled across the three training seeds for each probability and configuration. The reward and success curves show learning progress, although the performance becomes less stable as motion uncertainty increases. The experiment does not establish theoretical convergence to an optimal policy.

The reported evaluation rows pool the 100-episode results from the three training seeds. Per-seed means and standard deviations are not shown, so the results demonstrate performance across the three tested seeds but do not quantify between-seed variance. Robustness conclusions should therefore be limited to these tested runs and this fixed layout.

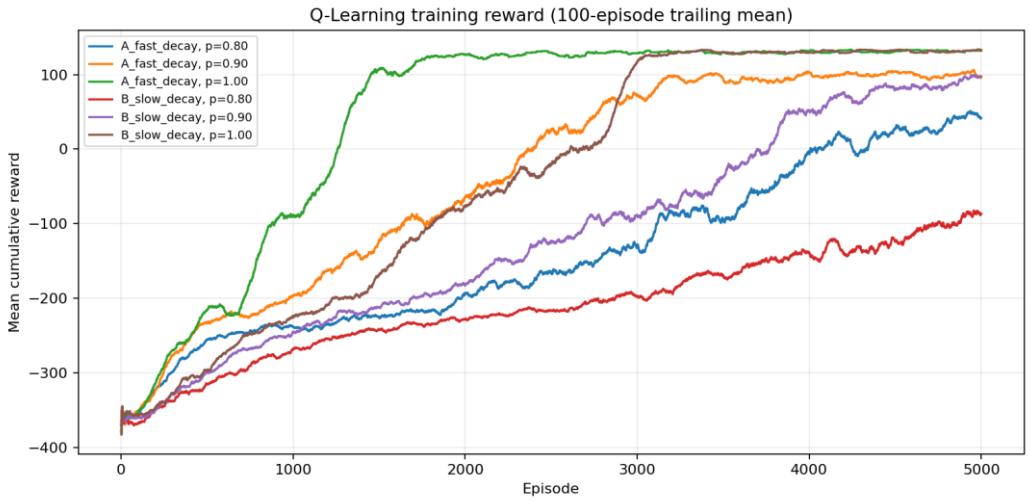


Figure 4. Q-Learning training cumulative reward against episode, using a trailing moving average.

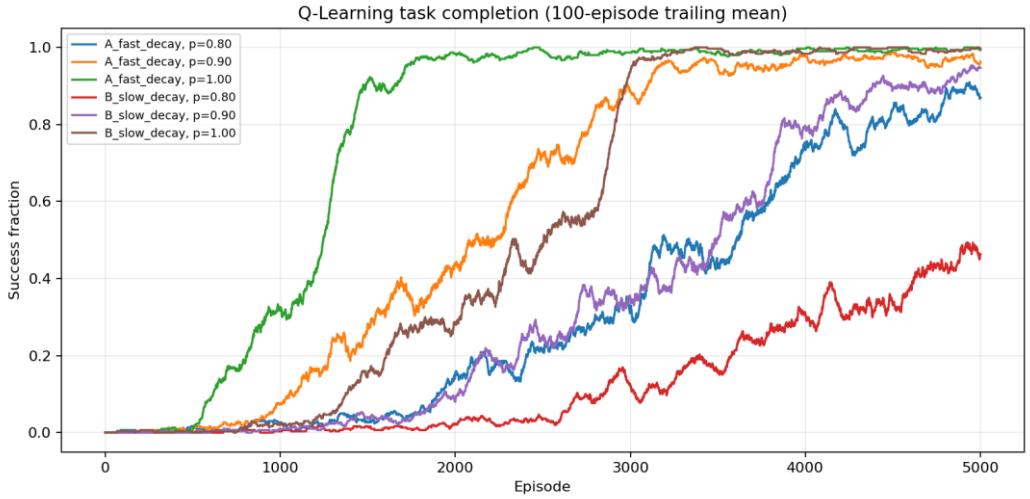


Figure 5. Q-Learning training success fraction against episode, using a trailing moving average.

After training, both configurations completed all greedy evaluations at p = 1.00. At p = 0.80, Configuration A achieved 88.67% success, while Configuration B achieved 49.00% success. The lower success rate under the 5,000-episode budget is consistent with the learning challenge created by a large time-aware state space and stochastic movement, although these experiments do not isolate the effect of either factor. Results across seeds should be interpreted as evidence from three training runs, not a guarantee for other maps or training conditions.

4.4 Greedy Policy and Evaluation

After training, the learned greedy policy selects the action with the highest Q-value at each observed state. Evaluation uses epsilon = 0 and does not update Q-values. Figure 6 displays policy arrows for p = 1.00, 0.90, and 0.80 at the three task stages and selected remaining-step slices. Arrows are restricted to states visited in training; unmarked cells must not be interpreted as having no possible action.

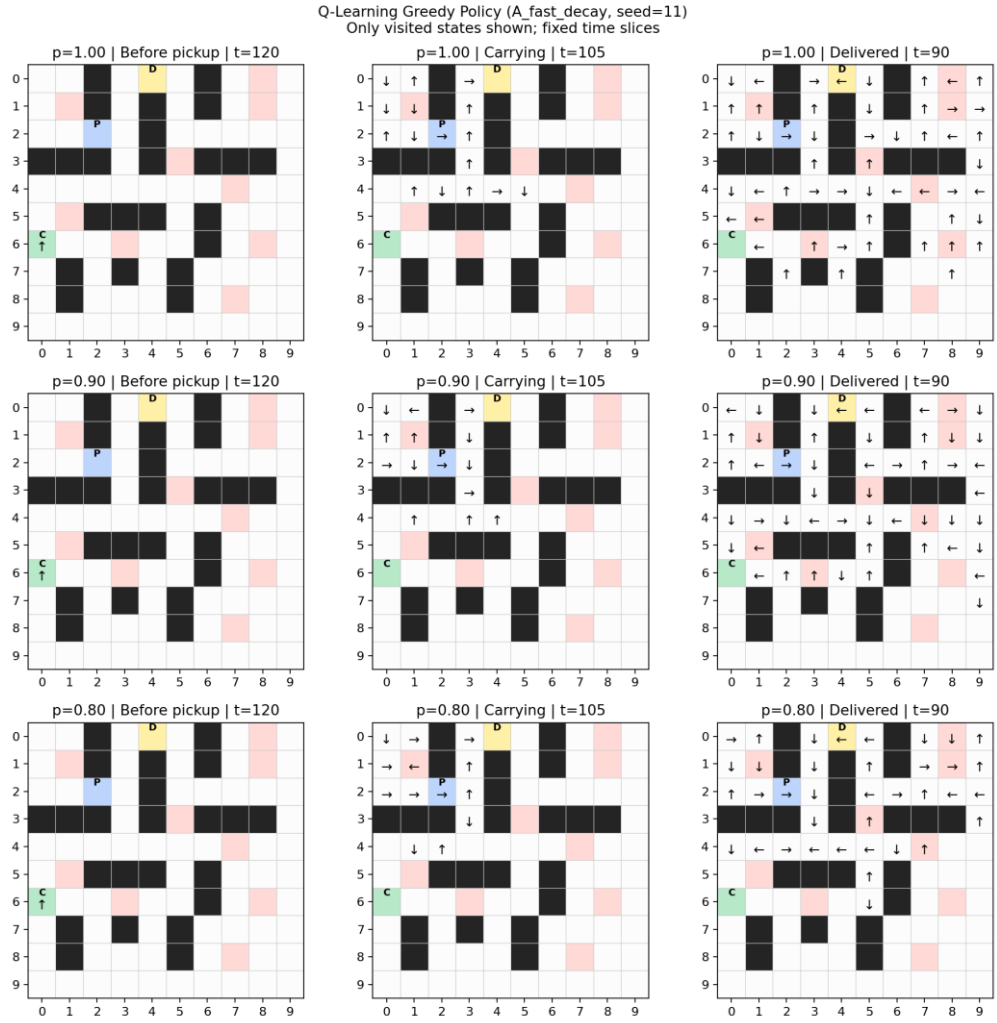


Figure 6. Q-Learning policy slices for the three movement probabilities and task stages; only visited states are shown.

For example, the first greedy evaluation of Configuration A, training seed 11, at p = 1.00, completed pickup, delivery, and return to the charging dock in 27 steps with cumulative reward 135.00. The corresponding recorded episode at p = 0.80 completed in 49 steps with reward 85.00. These are individual sampled runs, not the averages in Section 5.3. Figure 7 visualizes one sampled evaluation route per probability.

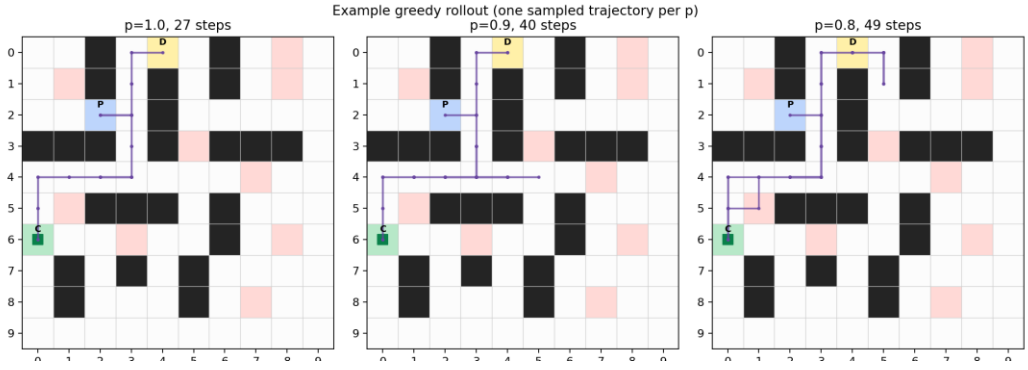


Figure 7. Example greedy evaluation trajectories under p = 1.00, 0.90, and 0.80.

5. Part 3: Stochastic Environment Experiments

5.1 Experimental Setup

To investigate motion uncertainty, the intended-action probability was set to p = 1.00, 0.90, and 0.80. The remaining probability was split equally between the two perpendicular directions. Both algorithms use the same warehouse layout (seed 42), five-dimensional state representation, 120-step horizon, terminal conditions, and reward definitions. Value Iteration uses $\gamma = 0.95$ and $\theta = 10^{-6}$; its policies are evaluated over 500 episodes per probability. Q-Learning Configuration A uses $\gamma = 0.95$ and B uses $\gamma = 0.99$; their evaluations use 100 episodes per trained seed. The evaluation sets are not paired.

5.2 Value Iteration Results under Different Motion Probabilities

p	Convergence sweeps	Average cumulative reward	Success rate	Average steps
1.00	2	143.00	100%	22.00
0.90	2	132.37	100%	25.30
0.80	2	118.08	100%	29.47

The average cumulative reward decreased as motion became less reliable, while the average number of steps increased. This indicates that unintended moves made task completion less efficient in the tested layout. All 500 evaluation episodes succeeded in each setting on this fixed layout; this result should not be interpreted as a guarantee for every possible layout.

5.3 Q-Learning Results under Different Motion Probabilities

The Q-Learning policies were evaluated with the same warehouse layout, reward rules, and intended-action probabilities as the Value Iteration experiments. Each table row combines 300 greedy evaluations, corresponding to 100 evaluation episodes for each of the three training seeds. Success requires pickup, delivery, and return to the dock within 120 steps. Average episode length includes both successful and unsuccessful episodes; an unsuccessful episode may continue until the 120-step limit.

Configuration	p	Average reward	Success rate	Average steps
A	1.00	137.00	100.00%	26.00
A	0.90	107.00	98.00%	37.92
A	0.80	52.05	88.67%	54.91
B	1.00	137.00	100.00%	27.00
B	0.90	106.84	96.67%	38.02
B	0.80	−66.46	49.00%	87.94

The average reward decreased and the number of steps increased as the intended-action probability decreased. Configuration A remained more reliable than B at p = 0.80 within the given training budget. These values come from the corrected five-component Q-Learning implementation and replace the earlier four-component results.

5.4 Comparative Performance under Different Probabilities

The Value Iteration results reported in Section 5.2 and the corrected Q-Learning results in Section 5.3 are summarized below. Configuration A uses $\gamma = 0.95$, matching the Value Iteration setting. Configuration B uses $\gamma = 0.99$ and serves as a separate hyperparameter experiment.

p	VI mean reward	QL A mean reward	VI success	QL A success
1.00	143.00	137.00	100%	100%
0.90	132.37	107.00	100%	98.00%
0.80	118.08	52.05	100%	88.67%

Value Iteration produced the higher reported average reward and success rate under each probability. However, Value Iteration was evaluated over 500 episodes per probability, while each Q-Learning row contains 300 evaluations across trained seeds. The rollout seeds were not established as identical, so this is a descriptive comparison rather than a paired statistical test. Direct state-by-state policy agreement also requires matching the five-component states and remaining-step values in both implementations.

5.5 Policy Visualization and Motion Uncertainty

5.5.1 Policy Visualization

Figures 1–3 present the Value Iteration policies for p = 0.80, 0.90, and 1.00, respectively, in Section 3.3. Figure 6 presents the learned Q-Learning policy slices for all three probabilities. The Q-Learning arrows show only states visited during training at the selected remaining-step slices; unmarked cells should not be interpreted as having no possible action.

5.5.2 Effect of Motion Uncertainty on the Preferred Action

The updated five-dimensional policies were compared at the same complete state and time horizon. The verified example is s = (5, 1, 1, 1, 96): the robot is at row 5, column 1, has collected and delivered the package, and has 96 steps remaining. The preferred action changes from Down at p = 1.00 to Left at p = 0.90 and p = 0.80, as shown below.

State (row, col, has_package, delivered, steps_remaining)	p = 1.00	p = 0.90	p = 0.80
(5, 1, 1, 1, 96)	Down	Left	Left

At (5, 1), the robot is on a congestion cell. With Down, the intended outcome is (6, 1); the perpendicular outcomes are Left to (5, 0) and Right to the shelf at (5, 2). A blocked move leaves the robot on the congestion cell, so that outcome earns $-1 - 3 - 10 = -14$. Its probability rises from 0.05 at p = 0.90 to 0.10 at p = 0.80. With Left, the three possible destinations are (5, 0), (4, 1), and (6, 1); all are passable and non-congested. As p decreases, Down becomes riskier, reducing its expected return and making Left preferable in the verified policies.

The congestion penalty is checked on the resulting cell after blocked movement is resolved. Therefore, attempting to enter a shelf while already on a congestion cell incurs both the invalid-move and congestion penalties, consistent with the supplied environment and Section 2.3.

6. Comparison and Discussion

6.1 Average Cumulative Reward and Success Rate

At p = 1.00, Value Iteration obtained an average reward of 143.00, compared with 137.00 for Configuration A and 137.00 for Configuration B. At p = 0.90, the averages were 132.37, 107.00, and 106.84, respectively. At p = 0.80, Value Iteration obtained 118.08, while Configuration A obtained 52.05 and Configuration B obtained −66.46. Value Iteration reported a 100% success rate at all three probabilities. Q-Learning success decreased to 88.67% for A and 49.00% for B at p = 0.80. These observations indicate lower measured Q-Learning performance for the available training budget, but the evaluation sets are not paired and the figures should not be interpreted as an exact estimate of algorithmic superiority across all conditions.

6.2 Convergence Behavior

Value Iteration uses the known model to compute expected state values. The five-dimensional finite-horizon implementation stopped after 2 sweeps for each p = 1.00, 0.90, and 0.80. In the first in-place Gauss–Seidel sweep, states are processed in ascending remaining-step order, so each successor at $\tau - 1$ has already been updated; this performs backward induction. The second sweep confirms that the maximum value change is below $\theta = 10^{-6}$. Q-Learning instead updates action values from sampled interactions and was trained for 5,000 episodes per configuration, p, and seed. Planning sweeps and training episodes are different units and are not directly comparable as computational costs.

6.3 Policy Differences and Impact of Motion Uncertainty

Both methods address the same five-dimensional task and represent its three task-progress phases. Value Iteration uses the known model to calculate an action for every valid state in each displayed time slice; Q-Learning estimates action values from sampled rewards, and Figure 6 shows only states visited during training. This is a qualitative similarity in task structure, not evidence of identical actions at matched states. The included compare_policies.py compares Q-Learning policies across p, with a visitation filter; it does not compare Q-Learning directly with Value Iteration. Because no complete matched-state action record for both algorithms is available, no numerical VI–QL action-agreement percentage is reported. Q-Learning's higher average episode lengths suggest lower overall efficiency in these evaluations, but unsuccessful episodes may reach 120 steps, so this metric is not the length of successful routes alone.

7. Conclusion

Both algorithms use five-dimensional states for robot position, task progress, and remaining time. Value Iteration plans with the known stochastic model, while Q-Learning learns from sampled interactions. In this fixed layout, lower intended-action probabilities generally reduced reward and increased episode length. At p = 0.80, Q-Learning Configurations A and B achieved success rates of 88.67% and 49.00%, respectively; Value Iteration achieved 100% in its 500-episode evaluations. Because the evaluation sets were not paired and sample sizes differed, these results are descriptive. More Q-Learning training and paired evaluation would strengthen the comparison.

8. Group Contributions and AI Usage Statement

8.1 Group Contributions

Member / role	Contribution
Lanhui Zhang (0465267)	MDP formulation, Value Iteration implementation, policy visualizations, and Value Iteration experiments.
Leyan Zeng (0480234)	Q-Learning implementation, training experiments, hyperparameter tests, and learned-policy evaluation.
Both members	Cross-algorithm comparison, discussion, final report integration, and final code/output review.

8.2 AI Usage Statement

We used OpenAI ChatGPT to help understand the assignment requirements, discuss the algorithms, check parts of our Value Iteration and Q-Learning code, and improve the wording of our report. We reviewed the code and results before submission.

Conversation links: https://chatgpt.com/share/6aef6e1b-27c8-83ea-ad80-a71f47888e9a
https://chatgpt.com/share/6aef6095-5a6c-83ea-a53f-07abde8c9a5f